

Gamified ML: The Complete Project Report

A plain-English walkthrough of everything we did, why we did it, and what we learned. Written for someone new to data science.

How to read this report

This report follows the project from start to finish, in the order we did it. Each section has three parts: **what we did**, **why**, and **the concepts you need to understand it**. When a technical word first appears, it is explained right there. There is also a glossary at the very end.

You do not need any maths. If you can follow a cooking recipe, you can follow this.

Contents

1. The big picture: what is this project?
 2. Machine learning in five minutes
 3. The data
 4. Stage 1: Exploring the data (EDA)
 5. Stage 2: Cleaning and building features
 6. Stage 3: Splitting the data and training the first models
 7. Stage 4: Improving the models
 8. Stage 5: Opening the black box (interpretation)
 9. Stage 6: The Metacritic experiment
 10. The full list of decisions we made
 11. What the project shows, and what it doesn't
 12. Where to go next
 13. Glossary
-

1. The big picture: what is this project?

You started with a large table of video games. Each row is one game, and the columns describe it: its name, release date, genres, platforms, developers, and a **rating** from 0 to 5 given by players.

The question we chose:

Can we predict whether a game is "highly rated" (rating 4.0 or above) using only information available about the game itself, and which pieces of information matter most?

This is a classic beginner machine learning project because it has all the important ingredients: messy real data, a clear goal, traps to avoid, and results you can explain.

The short answer we ended up with:

- Yes, but only modestly. Our best models are about **twice as good as random guessing**.

- The strongest clues are **when the game was released** and **what kind of game/publisher/platform it is**, not anything about the game's actual quality.
 - The models can't see whether a game is truly good, so they miss "hidden gems" and over-rate some old classics.
-

2. Machine learning in five minutes

Before the project, five ideas.

2.1 What is machine learning?

Normally, to make a computer do something, you write exact rules: "if the game is from 1998 and made by Nintendo, then say it's good." In **machine learning**, you instead show the computer many examples with the right answers, and it **works out the rules itself**.

Think of teaching a child to recognise dogs. You don't hand them a definition. You show them lots of pictures labelled "dog" and "not dog", and they figure out what dogs look like.

2.2 Features and target

- The **target** is the thing we want to predict. Ours is "is this game highly rated? yes (1) or no (0)."
- The **features** are the clues we give the model. Ours include release year, genres, platforms, and so on.

Each game becomes one row of clues plus one answer.

2.3 Classification

When the answer is a category (yes/no, cat/dog, spam/not spam), the task is called **classification**. When the answer is a number (like the exact rating 3.72), it's called **regression**. We chose classification because "is it good?" is a more natural question than "will it be exactly 3.72?".

2.4 Training and testing

If you study for an exam by memorising the exact answers to the practice questions, you'll ace the practice but might fail the real exam. Models do the same thing, and it's called **overfitting**: learning the quirks of the examples instead of the real patterns.

To catch this, we hide part of the data:

- The **training set** is what the model learns from.
- The **test set** is a hidden exam. We only ever ask the model to predict games it has never seen.

If the model does well on the hidden exam, it has really learned something.

2.5 The pipeline

Almost every ML project has the same shape:

1. **Look at the data** (exploration)
2. **Clean it and turn it into numbers** (feature building)
3. **Split it** into training and test
4. **Train** models

5. **Improve** them
6. **Understand** what they learned
7. **Report** honestly

Our six stages follow this exactly.

3. The data

The data is a big table (a **CSV file**) of **474,417 video games** with **27 columns**, apparently collected from the RAWG video game database around 2019-2020.

Important columns:

Column	Meaning
name	The game's title
released	Release date
genres, platforms, developers, publishers	Text lists, e.g. Action Indie (two genres separated by)
esrb_rating	Age rating (Everyone, Teen, Mature...)
rating	Average player rating, 0-5. This is what we base our target on.
ratings_count	How many players rated it
metacritic	Critic score, 0-100, but mostly missing
added_status_*	How many players own / beat / dropped / plan to play the game
playtime, achievements_count, website, ...	Other details

The first surprise: most games have no rating

A rating of 0 does **not** mean "terrible". It means **nobody rated it**. Only **11,994 of 474,417 games (2.5%)** have a real rating. Everything about the project follows from this: we can only learn from the tiny rated fraction.

4. Stage 1: Exploring the data (EDA)

EDA stands for *Exploratory Data Analysis*. It means looking at the data from many angles before building anything, like walking around a house before you buy it. The goal is to find problems, patterns and surprises early.

4.1 What we checked and found

Missing values. Many columns have gaps. Among all games: **metacritic** is 99% empty, ESRB rating 88%, website 86%, publishers 70%, genres 22%. Missing data is normal in real datasets. The question is always: *what do we do about it?* (We decide in Stage 2.)

The rating distribution. Among rated games, ratings pile up around 3 to 4. We plotted with a "log scale" for the full dataset because the 0s (unrated) are so numerous that they'd crush everything else.

Correlations. A correlation tells you whether two numbers tend to move together (+1 = perfectly together, -1 = perfectly opposite, 0 = unrelated). We originally computed correlations across all games, but that was misleading: 97.5% are zeros. Redoing it on rated games only gave much more honest numbers. For example, the Metacritic score correlates about 0.62 with the player rating, but the rating count only about 0.2.

4.2 Decision 1: Only trust games with enough ratings

The median rated game has just 19 ratings, and a quarter have 9 or fewer. A game with 3 ratings averaging 5.0 is probably luck, not quality. We plotted rating against rating count and saw a **funnel shape**: games with few ratings swing wildly between 1 and 5, and the swings narrow as the count grows.

This is the same reason you trust a restaurant with 2,000 reviews at 4.3 stars more than one with 2 reviews at 5 stars.

Minimum ratings	Games left
5	11,987
10 (chosen)	8,969
20	5,880
50	3,195

Trade-off: a high cut-off gives cleaner labels but fewer games to learn from. We picked 10 as the middle ground.

4.3 Decision 2: What counts as "highly rated"?

We had to draw a line. The choices:

Threshold	Share of games called "high"
3.5	54%
3.75	40%
4.0 (chosen)	26%
4.25	11%

A line at 3.5 makes the classes roughly balanced but "highly rated" would barely mean better than average. A line at 4.25 leaves too few positives to learn from. **4.0** felt like a genuinely good game and left 2,321 positive examples.

This creates **class imbalance**: 74% of games are "not high", 26% are "high". Imbalance matters a lot later.

4.4 The most interesting findings

Era matters enormously. About 62% of the rated games from 2000 are "highly rated", against about 13% for 2016. Why? Mostly **survivorship bias**: only the memorable old games are still being rated today, while the mid-2010s had a flood of low-quality indie and mobile releases. It isn't that games got worse. It's that the old ones we see are the survivors.

***Survivorship bias** is when you only look at the things that "survived" and wrongly conclude something about the whole group. (Famous example: studying only the WWII planes that returned from missions, and missing that the ones that got hit in other places never came back.)*

Genre and platform matter. Platformers (37%) and Fighting games (32%) score high; Indie, Casual and Massively Multiplayer sit around 10%. Retro consoles (N64, PlayStation, SNES) are 55-62% "high", against 21% for PC and 15% for Linux.

More genre tags, lower rating. Games with 1 genre are 38% "high", but games with 5 genres only 8%.

Small studios are numerous. There are 5,201 different developers and 3,214 publishers among the rated games, and the top 50 developers cover only 21% of entries. Most studios appear just once.

Metacritic is powerful but rare. It correlates strongly with rating, but only 39% of our rated games have one.

4.5 Data leakage: the most important trap

Some columns look like great predictors but are actually **consequences of the thing we're predicting**. This is called **data leakage**, and it's the most common way beginners fool themselves.

Examples in our data: `rating_top` (the most common star vote, correlation 0.83 with rating), `added_status_beaten` (0.56), `added_status_toplay` (0.49). Players "beat" or "own" games *after* they release, and partly *because* they liked them. If we let the model use these, it would look brilliant and be useless, like a student who gets the exam answers along with the questions.

A good rule: **for every feature, ask "would I actually know this at prediction time?"** If the answer is no, it's leakage.

5. Stage 2: Cleaning and building features

Models can only read numbers. This stage turns messy text into a clean numeric table.

5.1 Decision 3: Which columns can the model see?

We decided to build a **pre-release model**: it may only use facts about the game itself (genres, platforms, studios, release year, age rating), not anything created by players afterwards. So we removed 11 columns: `rating_top`, `ratings_count`, `reviews_count`, `suggestions_count`, `playtime` and all six `added_status_*`.

The alternative (a "post-release" model) would score much higher but wouldn't be honest or useful.

Metacritic is also post-release (critics review after launch) and mostly missing, so we set it aside for a separate experiment (Stage 6).

5.2 Decision 4: Bad rows

We dropped 91 rows: 89 with no release year, and 2 with dates after 2020 (unreleased games with ratings). That's about 1% of the data. We didn't guess the missing dates, because the release year is one of the strongest features and invented values would add false information. This left **8,878 games**.

5.3 Turning categories into numbers

A model can't read `Action||Indie`. Two techniques:

One-hot encoding. For a category like ESRB rating, create one 0/1 column per option: `esrb_Mature`, `esrb_Teen`, and so on. A game has a 1 in exactly one of them. Why not just number them 1, 2, 3? Because the model would think "Mature (3)" is *bigger than* "Teen (2)", which is meaningless.

Multi-hot encoding. The same idea when a game can belong to several categories at once (genres, platforms, developers, publishers). `Action||Indie` gets a 1 in both `genre_Action` and `genre_Indie`.

For missing ESRB ratings (58% of our games) we created a `Missing` category instead of guessing. We checked that "missing" itself wasn't informative (24.6% high when missing vs 27.7% when present).

5.4 Decision 5: Too many developers and publishers

We can't give each of 5,201 developers a column, since most appear once and the model would just memorise them. So we kept a column only for values that appear often enough:

- Genres and platforms: at least 30 games
- Developers and publishers: at least 10 games

Everyone else goes into an `all_rare` flag meaning "only small or unknown studios". This gave 20 genre, 37 platform, 207 developer and 176 publisher columns. About 53% of games have only rare developers and 36% only rare publishers.

Trade-off: a lower cut-off gives more columns and more chances of memorising individual studios; a higher one gives a cleaner but blunter model.

5.5 Decision 6: "Experience" features

We added `dev_experience` and `pub_experience`: the (logarithm of the) number of games that developer or publisher has made in the *whole* 474k dataset. A studio with 200 games is a different beast from a one-off hobbyist. It uses no ratings, so it isn't label leakage, and it gives the model a compact summary of studio size.

(A **logarithm** here just squashes huge ranges so 200 games vs 400 games doesn't look as dramatic as 1 vs 200.)

5.6 Other features we built

- `release_year`
- `n_genres`, `n_platforms` (how many of each)
- `has_website`, `has_achievements` (yes/no)

5.7 Safety checks

The code contains `assert` statements, which act as tripwires: they stop everything if a column name is duplicated, any value is missing, or a leakage column slipped in. It's cheap insurance against silent mistakes.

Result: a clean file of 8,878 games and 454 features, saved as `model_data.csv`.

6. Stage 3: Splitting the data and training the first models

6.1 Decision 7: How to split into training and test

There are two sensible ways.

Random split (80/20). Shuffle the games and put 80% in training, 20% in the test set. "Stratified" means both parts keep the same 26% share of "high" games. The training set has 7,102 games and the test set 1,776.

Time split. Train on games released up to 2015 and test on 2016-2020. This imitates real life: predicting the future from the past.

They answer different questions. Random tells you how well the model understands patterns in *this* dataset. Time tells you how well it would handle *new* games. We do both.

Notice something in the time split: the share of "high" games falls from 28.7% in training to 19.6% in the test period. The world changed between the two.

Golden rule: *the test set is touched only for final reporting. All decisions (choosing models, tuning) use only the training set. If you tune against the test set, it stops being a fair exam.*

6.2 Decision 8: How to measure success

Accuracy is a trap here. With 74% "not high" games, a model that always says "not high" is 74% accurate and completely useless. So we use better measures:

Precision means: *of the games the model flagged as "high", how many really were?* High precision = few false alarms.

Recall means: *of all the truly high games, how many did the model find?* High recall = few missed gems.

There's a tension: flag more games and you catch more good ones (recall up) but also more duds (precision down).

F1 is a single score that balances precision and recall.

ROC-AUC measures whether the model ranks good games above bad ones. Take a random good game and a random bad game: AUC is the chance the model gives the good one a higher score. So 0.5 means coin-flipping and 1.0 means perfect.

PR-AUC is similar but focuses on the "high" class, and it's harsher for imbalanced data. Its "no skill" baseline equals the share of positives (0.26 for us). Beating that line is what matters.

6.3 Decision 9: Handling imbalance

We set `class_weight='balanced'`, which tells the model that missing a rare "high" game is a bigger mistake than raising a false alarm. Without it, models tend to almost never predict "high". A side effect is that predicted probabilities run higher than the true 26% rate, so read them as scores, not literal chances.

6.4 Cross-validation

We needed a way to compare models without touching the test set. **Cross-validation** does it: split the training set into 5 parts, train on 4, score on the 5th, and rotate so every part gets a turn as the mini-exam. You get an average score plus a "±" telling you how much it wobbles.

6.5 The four models

Dummy. Always predicts the base rate. The floor everything must beat.

Logistic regression. Despite the name, it's a classifier. It gives every feature a weight ("Nintendo publisher: +", "PC: -"), adds them up, and converts the total into a probability. It's simple and easy to explain. Because features have different scales (year is about 2010, others are 0/1), we rescale them first (`StandardScaler`).

Random forest. Builds hundreds of **decision trees** (flow charts of yes/no questions like "released before 2005? on PC? Indie?") and lets them vote. Each tree sees a random slice of data, which cancels out individual mistakes.

Gradient boosting. Also uses trees, but builds them one after another, each one fixing the mistakes of the previous ones. Usually the most accurate of the classic methods.

6.6 What we found

Cross-validation on training data:

Model	ROC-AUC	PR-AUC
Dummy	0.500	0.260
Logistic Regression	0.783	0.552
Random Forest	0.815	0.609
Gradient Boosting	0.812	0.599

All real models are far above the dummy.

Final test, random split: all three about 0.81-0.83 AUC, PR-AUC about 0.59-0.62.

Final test, time split: all three drop to about **0.69-0.72 AUC** and 0.38-0.40 PR-AUC (against a no-skill 0.20). Interestingly, the simple logistic regression now **beats** the fancy tree models. Our reading: trees learn era-specific patterns (this studio, that year) that don't carry over to newer games, while the simple model generalises better.

This gap is the single most important finding in the project: **a model that looks good on shuffled data can look much weaker when asked to predict the future.** It's called **distribution shift**: the world the model is tested on differs from the world it learned from.

6.6b Two experiments that taught us something

Leakage demo. We trained a model *with* the leaky columns to see what happens. Result: **0.94 ROC-AUC, 0.85 PR-AUC**. Spectacular, and totally worthless. This is what leakage looks like, and why Decision 3 mattered.

Ablation. "Ablation" means removing something to see what breaks. We removed one feature group at a time and re-scored. Removing `release_year` cost about 0.015 AUC, genres 0.009, platforms 0.005; everything else 0.004 or less, and the 207 developer columns cost nothing. Nothing collapsed, because the features overlap: when you remove one, the others compensate. So "small drop" doesn't mean "useless", just "redundant".

6.7 Decision 10: Dropping `in_series`

We had flagged `in_series` (is the game part of a series?) as a grey area: a game may have sequels *because* it was popular. Removing it cost only about 0.003 AUC, less than the noise between cross-validation folds. So we dropped it for cleanliness. The result: **453 features**.

7. Stage 4: Improving the models

7.1 Decision 11: Which features for which model

Removing the developer/publisher columns did nothing to gradient boosting (0.830 vs 0.831 AUC) but made it about 5× faster. Logistic regression, though, needed them (dropping them cost about 0.03 AUC). So:

- Logistic regression gets all 453 features.
- Gradient boosting gets a lean 72-feature set (the big one-hot developer/publisher columns removed, but experience and all_rare flags kept).

Why the difference? A simple linear model can't combine features, so it leans on lots of small clues. Trees build their own combinations ("old AND Nintendo") so extra columns mostly add noise.

7.2 Hyperparameter tuning (Decision 12)

Hyperparameters are the model's settings that *you* choose (unlike the weights it learns). Think of the dials on an oven: temperature and time. Tuning means trying different settings and keeping the best.

- For logistic regression the main dial is **C**, the **regularisation strength**. Regularisation is a penalty that stops the model from over-trusting any single feature. Smaller **C** means a stronger penalty. With 453 features and only about 7,000 games, strong regularisation won: **C = 0.001**.
- For gradient boosting, we tried 10 random combinations of tree depth, learning speed, number of rounds and so on (a **random search**, cheaper than trying every combination).

We tuned using ordinary 5-fold cross-validation, optimising PR-AUC.

7.3 The disappointing (and useful) result

Tuning barely helped:

Model	Before	After (test PR-AUC)
Logistic Regression	0.591	0.602
Gradient Boosting	0.616	0.607

Tuned logistic regression gained about 0.01; boosting didn't improve at all. This tells us the models are **near their ceiling**: more dial-twiddling won't help; better *information* would.

We also learned that **ordinary cross-validation is over-optimistic for the future**. On pre-2016 games it predicted PR-AUC around 0.61-0.65, but the same models scored only about 0.37-0.41 on 2016+ games. Time-aware cross-validation predicted about 0.47-0.50, closer to the truth. Moral: use cross-validation to *compare* models, not to promise real-world scores.

7.4 Decision 13: The threshold

A model outputs a probability, e.g. 0.63. The **threshold** decides where "yes" begins. Lower it and more games are flagged (higher recall, lower precision); raise it and the reverse.

We chose the threshold using only training data (out-of-fold predictions), aiming for the best F1. The surprise: the best threshold was about 0.46-0.49, nearly the default 0.5. That's because **class_weight='balanced'** had already centred the probabilities. Without balanced weights, the best threshold would have been far lower.

Trade-offs for gradient boosting on the random test set (1,776 games, 461 truly high):

Threshold	Games flagged	Precision	Recall	F1
0.3	1,034	0.40	0.90	0.56
0.5	681	0.52	0.77	0.62
0.6	527	0.59	0.67	0.63
0.7	368	0.62	0.50	0.55
0.8	203	0.66	0.29	0.40

Which threshold is "right" depends on the purpose. "Find hidden gems" wants high recall; "recommend only sure things" wants high precision.

A **confusion matrix** at the chosen threshold: 356 good games correctly caught, 105 missed, 333 false alarms, 982 correctly rejected. In plain terms: the model flags about 38% of games, finds about 77% of the good ones, and about half of what it flags is really good, compared with 26% by random picking. That's roughly **2x better than guessing**.

7.5 Decision 14: Keep both models

Logistic regression is the interpretable one (a weight per feature) and holds up best on the time split. Gradient boosting is the more accurate one on the random split. Each has a job, so we saved both.

8. Stage 5: Opening the black box

Now the original question: *what drives high ratings?*

Important warning: everything here is **association, not causation**. "Older games are rated higher" doesn't mean age makes a game good. It reflects who survives and who bothers to rate.

8.1 Two ways to read a model

Logistic regression weights. Just read them. Positive weight = pushes toward "high"; negative = toward "not high". Because we standardised the features, weights are comparable. Since strong regularisation shrinks them, compare ranks, not sizes.

Pushing toward **high**: Nintendo as publisher, Nintendo Switch, big-publisher experience, Mature rating, Blizzard, BioWare, FromSoftware, Naughty Dog, Platformer genre.

Pushing toward **not high**: *has_achievements* (strongest), later release year, Indie, more genres, PC, Action, Casual, Sports, Massively Multiplayer.

(Achievements aren't bad for games. They're a stand-in for "modern, mass-market" here.)

Permutation importance. For models with no simple weights, shuffle one feature's values randomly and see how much the score drops. If shuffling barely matters, the model didn't need that feature; if the score collapses, it did. It's like removing one ingredient from a recipe to see if anyone notices.

8.2 Decisions 15 and 16: How to do it fairly

- **Measure on the test set**, not the training set. On the training set the model has memorised things, which rewards features good for memorising (like rare developer names).

- **Shuffle related features together as groups**, because our features overlap. If you shuffle `plat_PC` alone, the model just leans on `has_achievements` and `release_year`, which say something similar, so the drop looks small. Shuffling *all platforms together* gives the honest picture.

8.3 The key lesson: two models, same answer, different routes

Feature group	Gradient Boosting relies on it	Logistic Regression relies on it
release_year	0.091	0.003
genres	0.067	0.022
platforms	0.064	0.027
experience (dev/pub)	0.060	0.001
website + achievements	0.050	0.021
developer flags	0.024	0.050
publisher flags	0.001	0.067

Gradient boosting reads the era straight off `release_year`. Logistic regression barely touches it, and instead infers the era from publishers and platforms (Nintendo, PlayStation and Capcom are old-school signals). Same information, different door.

So "importance" depends on the model when features overlap. Don't say "release year is exactly this important". Say "era-related information is the dominant signal, and both models find it." This is a real-world lesson that catches out many experienced people.

8.4 What the curves show

We asked: "what if every game were released in year X?" and averaged the prediction. For gradient boosting, the score falls from about 65% (year 2000) to about 36% (2008) and 34% (2015), then rises to about 51% (2020). This matches the survivorship pattern from Stage 1. (Read the *shape*, not the heights, because balanced weights inflate the probabilities.) Predicted "high" also falls with more genre tags: 44% at 1 genre to 30% at 5.

8.5 Where the model is wrong

Looking at the most confident errors is one of the most educational things you can do.

False alarms (predicted high, actually below 4.0): beloved retro console games such as Mega Man 6 (3.97), Pokémon Snap (3.97) and Pilotwings 64 (3.87). They fall just short of the cut-off. These are near-misses, not real mistakes. The threshold is a hard line drawn through a smooth quality scale.

Misses (actually high, predicted low): indie darlings such as Overcooked (4.09), Nidhogg (4.06), Flower (4.02) and Star Control: The Ur-Quan Masters (4.20). To the model, these look like every other small-studio PC game. **Nothing in our columns says "this game is brilliant"**. That's the honest ceiling of a metadata-only model.

8.6 The answer to the original question

Based on metadata: **older, console-era games from big, established publishers and studios are rated higher than modern PC, indie, casual and multi-genre games.** Genre matters a bit. But the metadata explains only about twice as well as chance.

9. Stage 6: The Metacritic experiment

9.1 The question

Metacritic scores (critics' scores from 0-100) correlate strongly with player ratings, but they exist for only 3,465 of our 8,878 games (39%), and critics only score *after* release. Question: **how much would a critic score add?**

9.2 Decision 18: Keep it out of the main model

Since Metacritic is a post-release fact, adding it would turn our "predict before release" model into a "predict after critics have reviewed" model. That's fine, but a different question. So we kept the main model pure and treated Metacritic as a separate experiment.

9.3 Decision 19: How to handle the missing values

Two approaches, both tried:

1. **Subset only:** use just the 3,465 games that have a score, and compare models with and without it (using repeated cross-validation, since the subset is small).
2. **Whole dataset:** let gradient boosting treat "missing" as its own signal (it can handle gaps natively), and add a `has_metacritic` flag.

9.4 Results

On the 3,465-game subset (no-skill PR-AUC = 0.278):

Setup	ROC-AUC	PR-AUC
Metacritic score alone (one column, logistic regression)	0.839	0.684
Metadata only, no Metacritic (gradient boosting)	0.795	0.604
Metadata + Metacritic (gradient boosting)	0.880	0.758
Logistic regression, metadata only	0.791	0.617
Logistic regression, metadata + Metacritic	0.855	0.707

On the whole dataset (random split), gradient boosting improved from 0.828 to 0.871 ROC-AUC and from 0.603 to 0.691 PR-AUC. On just the test games that have a score, it jumped from 0.804 to 0.896 AUC.

9.5 What it means

- **One number (Metacritic) beats all 72 metadata columns combined.** A critic's verdict is much more informative about quality than anything about the game's category.
- It confirms the earlier conclusion: our metadata simply doesn't capture how good a game is. Metacritic does, in part.
- The catch: it isn't available before release and covers only 39% of games. So it can't rescue a "predict early" model, but it shows exactly where the missing information lives.

10. The full list of decisions

#	Decision	What we chose	Effect
1	Minimum ratings	10	Cleaner labels vs fewer games
2	"High" threshold	4.0	Meaningful "good" vs enough positives
3	Allowed columns	Pre-release only	Honest score vs inflated fake score
4	Bad rows	Dropped 91	Avoid invented dates
5	Rare categories	10/10/30/30 minimums	Model size vs memorising
6	Studio size	Experience features	Compact signal without extra columns
7	Split	Random + time	Development view + realistic view
8	Metrics	ROC-AUC, PR-AUC, F1	Not fooled by imbalance
9	Imbalance	Balanced class weights	Catch more good games
10	<code>in_series</code>	Dropped	Removes leakage risk
11	Features per model	453 vs 72	Speed and fit to each model
12	Tuning	5-fold CV on PR-AUC	Standard, but optimistic for the future
13	Threshold	F1 rule (about 0.5)	Balance of precision/recall
14	Final models	Keep both	Interpretable + accurate
15	Importance data	Test set	Avoids rewarding memorisation
16	Importance style	Grouped	Honest with overlapping features
17	Trust	Agreement between models	Avoids over-reading one model
18	Metacritic	Separate experiment	Keeps the main model honest
19	Missing Metacritic	Subset + native missing	Two views of the same question

11. What the project shows, and what it doesn't

What it shows

1. **Real data is mostly missing.** Only 2.5% of games had a usable rating.
2. **Leakage inflates scores massively** (0.94 AUC) and is easy to fall into.
3. **Split method changes the story.** 0.83 AUC on shuffled data becomes about 0.70 on predicting the future.
4. **Simple models can beat complex ones**, especially when the world shifts.
5. **Tuning has limits.** When you plateau, you need better information, not better dials.
6. **Feature importance is slippery** when features overlap.
7. **Error analysis reveals limits** that metrics can't (the indie gems, the retro near-misses).

What it doesn't show

- **Causation.** We found associations only.
- **Anything about all games.** We modelled roughly 2% of the raw table: games that people bothered to rate at least 10 times.
- **Current games.** The data is a 2019-2020 snapshot, and recent games have too few ratings.
- **Game quality.** The columns don't describe gameplay, story or polish.

12. Where to go next

- **Text from game names.** Markers like "Remastered", "II", "HD" or "(itch)" may capture some of the indie/classic split cheaply.
 - **Regression.** Predict the raw 0-5 rating instead of high/not-high, which avoids the artificial 4.0 cliff.
 - **Try other cut-off years** for the time split to see how the drop changes.
 - **Add outside data** (reviews, sales, tags) that actually describes quality.
 - **Learn more:** the scikit-learn user guide, the free "Kaggle Learn" micro-courses, and Andrew Ng's introductory machine learning course are all good next steps.
-

13. Glossary

Ablation. Removing a piece of the model or data to see how much it mattered.

Accuracy. Share of predictions that are correct. Misleading when one class is much bigger than the other.

Association vs causation. Two things moving together doesn't mean one causes the other.

Class imbalance. When one outcome is much rarer than the other (26% high vs 74% not).

Class weight. A setting that tells the model that mistakes on the rare class are costlier.

Classification. Predicting a category (yes/no) rather than a number.

Confusion matrix. A 2x2 table of correct and incorrect predictions: true positives, false positives, false negatives, true negatives.

Correlation. A number from -1 to +1 saying how strongly two things move together.

Cross-validation. Splitting the training data into parts and taking turns being the mini-exam, to score a model without touching the test set.

CSV. A simple text file holding a table.

Data leakage. Letting the model see information it wouldn't have at prediction time, often because it is a consequence of the answer.

Decision tree. A flow chart of yes/no questions that ends in a prediction.

Distribution shift. When the data the model is used on differs from the data it learned from (e.g. newer games).

EDA (Exploratory Data Analysis). Looking at the data from many angles before modelling.

F1 score. A single number that balances precision and recall.

Feature. One clue the model is given (release year, is-on-PC, etc.).

Gradient boosting. Building many trees in sequence, each correcting the previous ones' errors.

Hyperparameter. A setting you choose before training (like the oven temperature), as opposed to what the model learns.

Logistic regression. A simple classifier that gives each feature a weight and converts the total into a probability.

Multi-hot / one-hot encoding. Turning categories into 0/1 columns, one per option.

Overfitting. Memorising the training examples' quirks instead of learning general patterns.

Permutation importance. Shuffling a feature and measuring how much the score drops.

PR-AUC. A summary of the precision-recall trade-off across all thresholds; the baseline is the share of positives.

Precision. Of the games flagged "high", the share that really are.

Random forest. Many decision trees, each trained on random parts of the data, voting together.

Recall. Of the truly high games, the share the model found.

Regularisation. A penalty that stops a model from relying too heavily on any one feature.

Regression. Predicting a number rather than a category.

ROC-AUC. The chance the model ranks a random positive above a random negative (0.5 = guessing, 1.0 = perfect).

Stratified split. A split that keeps the same class proportions in both parts.

Survivorship bias. Drawing conclusions only from the items that "survived" a filter.

Target. The thing you are trying to predict.

Test set. Data held back and used only for the final score.

Threshold. The probability above which the model says "yes".

Training set. Data the model learns from.

Tuning. Trying different hyperparameter settings and keeping the best.